Topic 1: Lambda calculus concepts

Question 1 - Study Guide P09

```
let x = 10
let f y = x + y
let g () =
    let x = 99
    f 2
What does g() return? What would it return under dynamic scope?
Show a mutable-global simulation that restores the old value even if f throws.
```

Question 2 - Mock Midterm Q4

Write an F# program confirming that F# uses static scoping rather than dynamic scoping. Explain the different results the two rules would produce.

Question 3 - Scary Test Q2

A language called C** has familiar syntax:

```
int x = 1;
int f(int y) { return y+1; }
void MAIN() {
    int x = 2;
    stdout << f(x) << endl; // prints 3
}
```

Devise one C** program whose behavior differs under static and dynamic scoping. Explain how to interpret its output.

Question 4 - Lambda Quiz Q2 (PDF p. 1)

Determine if the following λ-terms are alpha-equivalent and explain why in each case.

(a)

```
λxλy.x + y        and        λyλx.y + x
```

(b)

```
λxλy.(y x) z    and        λyλz.(z y) x
```

Question 5 - Lambda Quiz Q7 (PDF p. 3)

What is the Type Soundness Theorem? What does it state?

Question 6 - F# Quiz Q2 (PDF p. 4)

Determine the output of the following fragment and explain why.

```
let x = 1
let f y = x+y
let g() =
    let x = 2
    printfn "%d" (f 0)
g()
```

Topic 2: F# functions and recursion

Question 1 - Study Guide P03

```
let f (x,y) = x+y
let g x y = x+y
let h = g 7
Give the types of f, g, h. Evaluate h 5 and f (7,5).
Explain why f 7 and g (7,5) fail for these definitions.
```

Question 2 - Study Guide P04

```
Use F1=1, F2=1. Explain why the naive two-call implementation is not
tail-recursive. Write fib : int -> int using a tail-recursive helper.
Reject n < 1 using invalidArg. Give fib 1, fib 2, fib 7.
Compare the naive and accumulator versions' time complexity.
```

Question 3 - Study Guide P05

```
Which recursive calls are in tail position? Explain short-circuit evaluation.
(a) let rec sum xs = match xs with [] -> 0 | h::t -> h + sum t
(b) let rec all p xs = match xs with [] -> true | h::t -> p h && all p t
(c) let rec copy xs = match xs with [] -> [] | h::t -> h :: copy t
(d) let rec any p xs = match xs with [] -> false | h::t -> p h || any p t
```

Question 4 - Study Guide P07

```
Write howmany xs predicate without List.filter or List.length.
Example: howmany [-2;0;3;4;7] (fun x -> x > 0) = 3.
Give its time complexity and explain the accumulator invariant.
```

Question 5 - Mock Midterm Q2

Determine whether each function is tail-recursive and explain why.

a. As printed in the photo:

```
let rec reduce id op m =
    match m with
    | [] -> id
    | a::b -> reduce (op id a) b
```

Transcription note: the printed recursive call appears to omit the `op` argument. For the intended tail-recursion question, analyze the corrected call `reduce (op id a) op b`. Also identify the problem with the printed version.

b.

```
let rec f n = if n < 2 then 1 else n * f (n - 1)
```

Question 6 - Mock Midterm Q3

Write a tail-recursive F# function `howmany` that counts values in a built-in list satisfying a predicate.

```
howmany (fun x -> x % 2 = 0) [2;5;6;7;9] // returns 2
```

Question 7 - Scary Test Q6

Given:

```
type List<'T> = Nil | Cons of 'T * List<'T>
```

Write a tail-recursive function that finds a list's length.

Question 8 - F# Quiz Q1 (PDF p. 4)

Write a curried version of:

```
let g(x,y) = x+y+1
```

Question 9 - F# Quiz Q1b (PDF p. 4)

Define a function `Curry` that transforms a non-curried function into a curried one. Hint: the type of this function should be:

```
('a * 'b -> 'c) -> ('a -> 'b -> 'c)
```

Question 10 - F# Quiz Q4 (PDF p. 4)

Is the following function tail-recursive?

```
let factorial n = if n<2 then 1 else n*factorial(n-1)
```

Question 11 - F# Quiz Q5 (PDF p. 5)

Given the type of linked lists as:

```
type List<'T> = Nil | Cons of 'T * List<'T>;;
```

Define `forall`, which takes a predicate and a list and determines whether all elements of the list satisfy the predicate, returning `true` or `false`.

Question 12 - F# Quiz Q5b (PDF p. 5)

Is your `forall` tail-recursive?

Topic 3: Pattern matching and error handling

Question 1 - Study Guide P06

```
Write safeDiv : int -> int -> int option with None for a zero denominator.
Evaluate:
Some 3 |> Option.bind (fun y -> Some 14 |> Option.bind (fun x -> safeDiv x y))
Replace Some 3 with Some 0, then None. What changes?
Why use bind rather than map with a function returning an option?
```

Question 2 - Study Guide P08

```
Write tail-recursive find_first predicate xs, returning an option.
Evaluate it for even numbers in [3;7;8;10], and for [].
Explain why it should not evaluate the predicate after the first match.
```

Question 3 - Study Guide P11

```
Using the expr type below, implement eval : expr -> expr.
Propagate RuntimeError; integer division by zero yields RuntimeError.
Predict these results before running:
(a) Plus(Num 3, Times(Num 4, Num 2))
(b) Divide(Num 7, Num 2)
(c) Times(Num 0, Divide(Num 1, Num 0))
(d) Divide(RuntimeError, Num 2)

type expr =
| Num of int
    | Plus of expr * expr
    | Times of expr * expr
    | Divide of expr * expr
    | RuntimeError
```

Question 4 - Study Guide P21

```
Write reciprocalOfFirstEven xs using find_first and Option.bind.
Find the first even number, then calculate integer 100 / that number safely.
Predict outputs for [3;4;8], [1;0;4], and [1;3;5].
Do not skip a matching zero to look for a later nonzero number.
```

Question 5 - Mock Midterm Q10

Given:

```
let safediv (a:float) (b:float) =
    if b <> 0.0 then Some(a / b) else None
// float -> float -> float option
```

and two `float option` values a and b, explain what, if anything, is wrong with:

```
a |> Option.bind (fun x ->
    b |> Option.map (fun y -> safediv x y))
  |> Option.iter (fun z -> printfn "quotient is %.2f" z)
```

Give a corrected version.

Question 6 - Mock Midterm Q11

Write an equivalent F# program for the following Java expression tree implementation. Use a discriminated union and an evaluation function.

```
public interface expr {
    int eval();
}
class intnode implements expr {
    int val;
    public intnode(int v) { val = v; }
    public int eval() { return val; }
}
class plusnode implements expr {
    expr left; expr right;
    public plusnode(expr l, expr r) { left = l; right = r; }
    public int eval() { return left.eval() + right.eval(); }
}
class timesnode implements expr {
    expr left; expr right;
    public timesnode(expr l, expr r) { left = l; right = r; }
    public int eval() { return left.eval() * right.eval(); }
}
```

Question 7 - Scary Test Q7

Given two int option values a and b, explain what, if anything, is wrong:

```
a |> Option.bind (fun x -> b |> Option.map (fun y -> x+y))
  |> Option.iter (printfn "sum is %d")
```

Question 8 - Scary Test Q9

Given:

```
type Number = Integer of int | Rational of int * int

let times a b =
    match (a,b) with
    | (Integer x, Integer y) -> Integer (x*y)
    | (Integer x, Rational(n,d)) -> Rational(x*n,d)
    | (Rational(a,b), Rational(c,d)) -> Rational(a*c,b*d)
    | (Rational(n,d), Integer x) -> Rational(x*n,d)

let inverse a =
    match a with
    | Integer x when x <> 0 -> Some(Rational(1,x))
    | Rational(n,d) when n <> 0 -> Some(Rational(d,n))
    | _ -> None
```

Write divide, multiplying the first Number by the inverse of the second when it exists. Return an option. Do not use Option.get.

Question 9 - Scary Test Q9b

Write the divide function from Scary Test Q9 without pattern matching, using map/bind.

Question 10 - F# Quiz Q3 (PDF p. 4)

Explain the difference, if any, between the following fragments of code.

(a)

```
match m with
| a::b::c when (a=b) -> printfn "first two are the same"
| _ -> printfn "something else"
```

(b)

```
match m with
| a::b::c -> if (a=b) then printfn "first two are the same"
| _ -> printfn "something else"
```

If they are different, under what value of `m` will the difference show up?

Question 11 - F# Quiz Q6 (PDF p. 5)

Given:

```
let safediv a b = if b=0 then None else Some(a/b)
let opt = Some 2;
```

What is the value of:

```
opt |> Option.map (fun x -> x+x)
    |> Option.map (fun y -> safediv 12 y)
```

Question 12 - Options Quiz Q1a–Q1d (PDF p. 6)

Given:

```
open Option
let a = Some(8)
let b = Some(2)
let c : int option = None

let safediv x y = if y=0 then None else Some(x/y)
```

For each expression, give its value.

O01a (original Q1a)

```
a |> map (fun x -> x+1)
```

O01b (original Q1b)

```
c |> map (fun x -> x+1)
```

O01c (original Q1c)

```
a |> map (fun x -> b |> map (fun y -> x+y))
```

O01d (original Q1d)

```
a |> bind (fun x -> b |> map (fun y -> safediv x y))
```

Question 13 - Options Quiz Q2 (PDF p. 6)

Given:

```
type expr =
    | Val of int
    | Times of expr * expr
    | Divide of expr * expr
    | Neg of expr;;
```

Show how to write a function `eval` of type `expr -> Option<int>` that evaluates an expression into an option, avoiding division by zero.

Topic 4: Mutable closures, streams and comparison to OOP

Question 1 - Study Guide P10

```
let make n =
    let mutable x = n
    fun y -> x <- x+y; x
let a = make 10
let b = make 10
let c = a
Evaluate in order: a 2; a 3; b 4; c 1; b 0.
Which closures share state? Must a itself be declared mutable?
```

Question 2 - Study Guide P19

```
Write makeOdds(), limit n stream, takeWhile predicate stream, and
filterStream predicate stream. Use permanent termination after None.
A fresh odds stream emits 1,3,5,... . For independent fresh sources, predict:
(a) makeOdds() |> limit 3                        (five pulls)
(b) makeOdds() |> takeWhile (fun x -> x < 6)     (five pulls)
(c) makeOdds() |> filterStream (fun x -> x % 3 = 0) |> limit 3
Explain why filterStream must keep pulling after a failed predicate, and why
limit 0 should not pull the source at all.
```

Question 3 - Mock Midterm Q5

Write `makeswitch`, which creates a closure that toggles between true and false. Use a mutable closure and no data structures. Separate switches must maintain independent state.

```
let s1 = makeswitch false
let s2 = makeswitch true
printfn "%A" (s1()) // true
printfn "%A" (s1()) // false
printfn "%A" (s1()) // true
printfn "%A" (s1()) // false
printfn "%A" (s2()) // false
printfn "%A" (s2()) // true
```

Question 4 - Mock Midterm Q9

Given this F# stream:

```
let oddstream() =
    let mutable x = 1
    fun () -> x <- x + 2; Some(x - 2)
```

Write a stream modifier `limit` that limits a stream to the first n values, unless None occurs earlier. For example, `oddstream() |> limit 5` should represent 1,3,5,7,9.

A stream here is a function of type `unit -> 'a option`, not a built-in sequence. The modified stream should return None after it ends.

Question 5 - Mock Midterm Q12

Identify a distinct advantage of each expression tree approach in Mock Midterm Q11: Java classes and F# discriminated unions. Be specific to extending this expression tree program.

Question 6 - Scary Test Q1

Explain the difference between f and g and determine the output.

```
let f x0 =
    fun y ->
        let mutable x = x0
        x <- x+y
        x

let g x0 =
    let mutable x = x0
    fun y ->
        x <- x+y
        x

let f1 = f(2)
let g1 = g(2)
printfn "%d" (f1(1)+f1(1))
printfn "%d" (g1(1)+g1(1))
```

Question 7 - Scary Test Q8a

A possibly infinite stream has type unit -> 'T option and ends when it returns None:

```
let odds() =
    let mutable n = 1
    fun () ->
        if n > 99 then None
        else
            n <- n+2
            Some(n-2)
```

Write take_while, taking a predicate and a stream, without Option.get. It must work for any stream and predicate:

```
odds() |> take_while (fun x -> x < 10)
// represents 1 3 5 7 9
```

Question 8 - Scary Test Q8b

Using odds() from Scary Test Q8a, write foreach, taking an action of type 'T -> unit and a stream of type unit -> 'T option. Call the action on each value until the stream ends.

```
odds() |> foreach (printf "%d ")
// prints 1 3 5 7 9 11 ... through 99
```

Topic 5: Limitations of C

Question 1 - Study Guide P20

```
Why is this unsafe in GNU C, and invalid in standard C?
typedef int (*intfun)(int);
intfun K(int x) {
    int inner(int y) { return x; }
    return inner;
}
How does an F# closure avoid the captured-local lifetime problem?
```

Question 2 - Mock Midterm Q6

Is it possible to write the makeswitch function in Mock Midterm Q5 in (GNU) C? Explain why or why not, considering the same independent, escaping closure behavior and restrictions.

Question 3 - Scary Test Q1b

Which of the functions in Scary Test Q1 can be written in GNU C? Explain.

Topic 6: OOP essentials

Question 1 - Study Guide P14

```
class A {
    String f(Object x) { return "A:Object"; }
}
class B extends A {
    @Override String f(Object x) { return "B:Object"; }
    String f(String x) { return "B:String"; }
}
A a = new B();
B b = new B();
Predict a.f("hi"), b.f("hi"), b.f((Object) "hi").
Separate compile-time signature selection from runtime implementation selection.
```

Question 2 - Study Guide P16

```
Write an abstract Shape with area(), a virtual destructor, and a Square.
Demonstrate dynamic dispatch through a Shape reference to a stack Square.
Why does this refute the claim that all polymorphic objects require the heap?
```

Question 3 - Mock Midterm Q8

Explain why dynamic memory allocation is required for advanced object-oriented programming. Give an example. Be precise about runtime object creation and object lifetimes.

Question 4 - Mock Midterm Q13

Will this compile? Will it run? Describe what happens and explain why.

```
interface city {}
class NY implements city {}
class LA implements city {}
public class tourist {
    static void visit(NY ny) {
        System.out.println("go to a yankees game");
    }
    static void visit(LA ca) {
        System.out.println("go to a dodgers game");
    }
    public static void main(String[] args) {
        city[] Cities = { new NY(), new LA() };
        for (city c : Cities) visit(c);
    }
}
```

Question 5 - Scary Test Q5

Determine the output and explain:

```
class A {}
class B extends A {}

public class goodquestion {
    public int f(A x) { return 1; }
    public int f(B x) { return 2; }

    public static void main(String[] args) {
        A n = new B();
        goodquestion gq = new goodquestion();
        System.out.println(gq.f(n));
    }
}
```

Topic 7: Templates, generics and variance

Question 1 - Study Guide P01

```
Assume Dog <: Animal. Which can safely replace Dog -> Animal?
(a) Animal -> Dog (b) Dog -> Dog (c) Animal -> Animal
(d) Poodle -> Animal, where Poodle <: Dog. Explain each decision.
Why is an unrestricted mutable Box<Dog> not a Box<Animal>?
```

Question 2 - Study Guide P02

```
For each independent snippet, say: compile error, successful execution, or
runtime failure. Explain the exact operation that causes failure.
(a) Object[] xs = new String[2]; xs[0] = "hi"; xs[1] = 42;
(b) ArrayList<Object> xs = new ArrayList<String>();
(c) List<? extends Number> xs = new ArrayList<Integer>(); xs.add(3);
(d) List<? super Integer> xs = new ArrayList<Number>(); xs.add(3);
What static type can you safely use for xs.get(0) in (d)?
```

Question 3 - Study Guide P15

```
interface I { Object f(); }
class C implements I { public String f() { return "ok"; } }
Does it compile? If I instead declares void put(Object x), does
public void put(String x) alone implement it? Explain.
```

Question 4 - Mock Midterm Q7

For each program, determine whether it compiles and, if it does, whether it produces a runtime error. Explain each answer in 1–2 sentences. Assume each public class has its own correctly named file.

a.

```
public class lotsofobjects {
    public static void main(String[] args) {
        Object[] A = new String[10];
        A[0] = 3.5; // Doubles are also objects
        for (Object x : A) System.out.println(x);
    }
}
```

b.

```
public class A<T> {
    public T[] M = new T[10];
}
```

c.

```
import java.util.ArrayList;
public class waytoogeneric {
    public static void main(String[] args) {
        ArrayList<Object> L = new ArrayList<String>();
        L.add("abc");
    }
}
```

Question 5 - Scary Test Q4

Explain why this should or should not compile, regardless of whether Java actually allows it:

```
Object[] A = new String[4];
```

Topic 8: How generics are implemented

Question 1 - Study Guide P12

```
(a) Why is class Box<T> { static T value; } rejected?
(b) What is T erased to in class Box<T extends Number> { T value; }?
(c) Can you write new T() or new T[10] inside an ordinary generic class?
(d) Contrast Java generics and C++ templates without claiming C++ skips checks.
```

Question 2 - Study Guide P13

```
Given template<typename T> requires std::totally_ordered<T>
bool before(T a, T b) { return a < b; }
If requires is removed, must every use compile? Explain for int and for a
custom type with no operator<. How can constraints help overloaded templates?
```

Question 3 - Mock Midterm Q14

Will this C++ template class compile? Explain why or why not. Distinguish defining a template from instantiating and using its members.

```
#include <iostream>
template<typename T>
class A {
private:
    T x;
public:
    A(T y) { x = y; }
    void f() { x = 3; }
};
```

Question 4 - Scary Test Q3

Explain whether this class compiles:

```
class A<T> {
    static T x;
}
```

Uncategorized questions

Question 1 - Study Guide P17

```
int* p = new int(3);
int* q = new int(4);
p = q;
delete p;
delete q;
Identify the leak and double delete precisely. Is *q valid after delete p?
Show a corrected version using std::unique_ptr.
```

Question 2 - Study Guide P18

```
Distinguish scope from lifetime. Does assigning 9 to an existing int end that
int object's lifetime? Does std::move(s) itself destroy s? Explain why a pointer
to a local variable must not be returned for use after the function exits.
```

Question 3 - F# Quiz Q7 (PDF p. 5)

If an array is passed to a function, is it passed a copy or a reference?